

🎓 Viva Q&A — Gradio Frontend & UI (segT)

Focused on the **Gradio web interface**, UI components, event handling, and frontend code in `app.py`.

1. Gradio Basics

Q1. What is Gradio?

A. Gradio is an open-source Python library that lets you build **interactive web UIs** for ML models with minimal code. You define input/output components (images, sliders, JSON, plots) and wire them to a Python function — Gradio handles the HTML, JS, WebSocket communication, and server automatically.

Q2. Why Gradio over Flask/Streamlit/React?

A.

	Gradio	Flask	Streamlit	React
ML-native components	✅ Image, Plot, JSON built-in	❌ Manual	Partial	❌ Manual
Lines of code for UI	~70	~300+	~100	~1000+
HuggingFace Spaces	✅ Native deploy	❌	❌	❌
Queueing	✅ Built-in	Manual	Limited	Manual
Interactivity	Event-driven	Request-response	Top-down rerun	Full control

Gradio was chosen for **rapid prototyping** and **one-click cloud deployment** on HuggingFace Spaces.

Q3. How do you install Gradio?

A. `pip install gradio` — it's listed in `requirements.txt`. Current project uses it as `import gradio as gr`.

Q4. How do you launch a Gradio app?

A.

```
demo = create_demo()          # Returns a gr.Blocks instance
demo.queue().launch(share=False)
```

- `.queue()` enables request queueing for concurrent users.
- `.launch(share=False)` starts a local server at `http://127.0.0.1:7860`.
- Setting `share=True` would create a temporary public URL via Gradio's tunnel.

2. UI Structure (`app.py` Code)

Q5. What Gradio API mode does your app use — Interface or Blocks?

A. `gr.Blocks` — the more flexible, lower-level API. Unlike `gr.Interface` (which is a simple function→UI wrapper), `gr.Blocks` allows:

- Custom layouts with `gr.Row()` and `gr.Column()`.
- Accordions, tabs, multiple buttons.
- Multiple inputs/outputs wired to different functions.
- Fine-grained control over component placement.

```
with gr.Blocks(title="Terrain Safety Analysis") as demo:
    # ... all components defined here
```

Q6. Explain the layout structure of the UI.

A. The UI uses a **two-column layout**:

gr.Markdown (Title + Description)			
Column 1 (scale=1)	Column 2 (scale=2, wider)		
Image Input	HUD	Depth	Mask
Settings (Accordion)	3D Terrain Plot		
Mapping JSON (Accordion)	Safety Score Label		
Advanced (Accordion)	Detailed Stats JSON		
[Analyze] Button			

Code:

```
with gr.Row():
    with gr.Column(scale=1):      # Left panel – inputs
        ...
    with gr.Column(scale=2):      # Right panel – outputs (2× wider)
        ...
```

Q7. What does `scale=1` vs `scale=2` mean in columns?

A. The `scale` parameter controls **relative width**. Column with `scale=2` gets **twice** the horizontal space of `scale=1`. So the output panel (images, plots, stats) gets 2/3 of the width, and the input panel gets 1/3.

Q8. What is `gr.Accordion` and how is it used?

A. `gr.Accordion` creates a **collapsible section** in the UI. It visually groups related controls:

```
with gr.Accordion("Settings", open=True):      # Expanded by default
    model_selector = gr.Dropdown(...)
    opacity_slider = gr.Slider(...)

with gr.Accordion("Class Mapping (JSON)", open=False): # Collapsed by default
    mapping_editor = gr.Code(...)

with gr.Accordion("Advanced Features", open=True):
```

```
enable_depth = gr.Checkbox(...)
...
```

`open=True/False` controls whether the section starts expanded or collapsed.

3. Input Components

Q9. List all input components in the app.

A.

Component	Gradio Type	Variable	Purpose
Image upload	<code>gr.Image(type="pil")</code>	<code>input_image</code>	User uploads terrain image
Model selector	<code>gr.Dropdown</code>	<code>model_selector</code>	Choose B0 or B2
HUD opacity	<code>gr.Slider(0, 1)</code>	<code>opacity_slider</code>	Control overlay transparency
JSON editor	<code>gr.Code(language="json")</code>	<code>mapping_editor</code>	Edit safe/hazard class lists
Enable Depth	<code>gr.Checkbox</code>	<code>enable_depth</code>	Toggle depth estimation
Depth Model	<code>gr.Dropdown</code>	<code>depth_size</code>	Small or Base depth model
Depth Opacity	<code>gr.Slider(0, 1)</code>	<code>depth_opacity</code>	Depth overlay transparency
Enable Path	<code>gr.Checkbox</code>	<code>enable_path</code>	Toggle A* pathfinding
Show 3D	<code>gr.Checkbox</code>	<code>show_3d</code>	Toggle 3D visualization
Analyze button	<code>gr.Button(variant="primary")</code>	<code>run_btn</code>	Trigger processing

Q10. Why is `type="pil"` used for the image input?

A. `gr.Image(type="pil")` tells Gradio to pass the uploaded image as a `PIL.Image` object to the callback function. Other options are:

- `type="numpy"` → numpy array (H, W, 3)
- `type="filepath"` → path string to a temp file

We use `PIL` because:

- `SegformerFeatureExtractor` accepts PIL images directly.
- PIL provides `.size` (W, H) and easy conversion methods.
- All our utility functions (`create_hud` , `create_depth_overlay`) work with PIL.

Q11. What is `gr.Code` and why use it for the mapping editor?

A. `gr.Code` renders a **syntax-highlighted code editor** in the UI. We use it with `language="json"` to provide a proper JSON editing experience for the safe/hazard class mapping:

```
mapping_editor = gr.Code(
    value=json.dumps(DEFAULT_MAPPING, indent=2),
    language="json",
    label="Safe/Hazard Definition"
)
```

This allows users to **modify the safety definitions at runtime** — e.g., moving "snow" from hazard to safe for a ski-navigation use case.

Q12. Explain the `gr.Dropdown` component for model selection.

A.

```
model_selector = gr.Dropdown(
    choices=list(MODELS.keys()),          # ["SegFormer B0 (Fast)", "SegFormer B2 (Balanced)"]
    value="SegFormer B0 (Fast)",          # Default selection
    label="Model Version"
)
```

The dropdown passes the **selected key string** to the callback. Inside `process_image()`, this key is mapped to the actual HuggingFace model ID:

```
MODELS = {
    "SegFormer B0 (Fast)": "nvidia/segformer-b0-finetuned-ade-512-512",
    "SegFormer B2 (Balanced)": "nvidia/segformer-b2-finetuned-ade-512-512"
}
model_name = MODELS[model_key] # Resolves human-readable name → model ID
```

Q13. How does the `gr.Slider` work?

A.

```
opacity_slider = gr.Slider(0, 1, value=0.4, label="HUD Opacity")
```

- `0, 1` → min and max values.
- `value=0.4` → default position.
- Returns a **float** to the callback.
- Used 3 times: HUD opacity (0.4), depth opacity (0.5), and implicitly constraining visual blend strength.

Q14. How does `gr.Checkbox` work?

A.

```
enable_depth = gr.Checkbox(label="Enable Depth", value=False)
```

Returns a **boolean** (`True` / `False`) to the callback. Default is `False` (unchecked). Three checkboxes control optional features: depth estimation, pathfinding, and 3D view. These are independent toggles — users can enable any combination.

4. Output Components

Q15. List all output components.

A.

Component	Gradio Type	Variable	Displays
HUD overlay	<code>gr.Image(type="pil")</code>	<code>output_hud</code>	Green/red safety overlay + path
Depth overlay	<code>gr.Image(type="pil")</code>	<code>output_depth</code>	Depth heatmap overlay
Raw mask	<code>gr.Image(type="pil")</code>	<code>output_mask</code>	Integer segmentation mask

3D terrain	gr.Plot	output_3d	Interactive Plotly 3D mesh
Safety score	gr.Label	score_display	"Safety Score: 73.21%"
Detailed stats	gr.JSON	output_json	Full stats dictionary

Q16. What is `gr.Plot` and how does it render Plotly figures?

A. `gr.Plot` renders **interactive Plotly figures** directly in the browser. When the callback returns a `plotly.graph_objects.Figure` object, Gradio serializes it to JSON and renders it using Plotly.js on the client side. Users can rotate, zoom, and pan the 3D terrain mesh interactively.

Q17. What is `gr.JSON` and what does it display?

A. `gr.JSON` renders a **formatted, collapsible JSON viewer** in the UI. Our callback returns a dictionary:

```
{
  "safety_score": 73.21,
  "safe_pixels": 195432,
  "hazard_pixels": 72104,
  "total_pixels": 267536,
  "safe_percentage": 73.05,
  "hazard_percentage": 26.95,
  "class_counts": {"grass": 120000, "water": 50000, ...},
  "top1_class": "grass",
  "mean_confidence": 0.8734
}
```

Gradio automatically renders this as a pretty-printed, expandable JSON tree.

Q18. What is `gr.Label` ?

A. `gr.Label` displays a **simple text label**, typically used for classification results or scores. We use it to show "Safety Score: 73.21%" prominently. It's more visually prominent than plain text.

5. Event Handling & Callback

Q19. How is the button click wired to the processing function?

A.

```
run_btn.click(
    process_image,          # The callback function
    inputs=[                # 9 input components
        input_image, model_selector, opacity_slider,
        mapping_editor, enable_depth, enable_path,
        show_3d, depth_size, depth_opacity
    ],
    outputs=[               # 6 output components
        output_hud, output_depth, output_mask,
        output_json, score_display, output_3d
    ]
)
```

When the user clicks "Analyze Terrain", Gradio:

1. Reads the current values of all 9 input components.
2. Calls `process_image(image, model_key, opacity, mapping_json_str, enable_depth, enable_path, show_3d, depth_model_size, depth_opacity)`.
3. The function returns 6 values (in order).
4. Each return value is sent to the corresponding output component.

Q20. What is the function signature of `process_image()` ?

A.

```
def process_image(image, model_key, opacity, mapping_json_str,
                  enable_depth, enable_path, show_3d,
                  depth_model_size, depth_opacity):
```

Parameters (matched 1:1 with `inputs` list):

Parameter	Type	From Component
<code>image</code>	<code>PIL.Image</code>	<code>input_image</code>
<code>model_key</code>	<code>str</code>	<code>model_selector</code>
<code>opacity</code>	<code>float</code>	<code>opacity_slider</code>
<code>mapping_json_str</code>	<code>str</code>	<code>mapping_editor</code>
<code>enable_depth</code>	<code>bool</code>	<code>enable_depth</code>
<code>enable_path</code>	<code>bool</code>	<code>enable_path</code>
<code>show_3d</code>	<code>bool</code>	<code>show_3d</code>
<code>depth_model_size</code>	<code>str</code>	<code>depth_size</code>
<code>depth_opacity</code>	<code>float</code>	<code>depth_opacity</code>

Returns: `(hud_image, depth_overlay, mask_colored, json_output, score_text, fig_3d)` — 6 values for 6 output components.

Q21. What happens if the user doesn't upload an image?

A.

```
if image is None:
    return None, None, None, None, None, "Please upload an image.", None
```

The function returns `None` for all image/plot outputs and a text message for the score label. Gradio handles `None` gracefully — it simply shows empty/blank components.

Q22. How is the JSON mapping parsed safely?

A.

```
try:
    mapping_config = json.loads(mapping_json_str)
except:
    mapping_config = DEFAULT_MAPPING
```

If the user types invalid JSON in the editor, `json.loads()` throws an exception — the bare `except` catches it and falls back to the default mapping. This prevents the app from crashing on malformed user input.

Q23. How does the callback handle optional features (depth, path, 3D)?

A. Conditional logic using the boolean checkboxes:

```
if enable_depth or enable_path:
    depth_pipe = model_utils.load_depth_model(...)
    depth_map_norm = model_utils.estimate_depth(image, depth_pipe)

if enable_path and depth_map_norm is not None:
    path_coords = model_utils.compute_path(safety_mask, depth_map_norm)
elif enable_path:
    path_coords = model_utils.compute_path(safety_mask, None)

if enable_depth and depth_map_norm is not None:
    depth_overlay = model_utils.create_depth_overlay(...)

if show_3d and depth_map_norm is not None:
    fig_3d = model_utils.create_3d_terrain(...)
```

Each feature is computed only if its checkbox is checked. If depth fails but pathfinding is enabled, pathfinding still runs using the safety mask alone (without slope penalties).

6. Examples & Samples

Q24. How are example images provided?

A.

```
gr.Examples(
    examples=[["assets/sample.jpg"]],
    inputs=input_image
)
```

`gr.Examples` renders clickable example thumbnails below the UI. When clicked, the example image is loaded into `input_image`. The `examples` parameter is a list of lists — each inner list provides values for the specified `inputs` components.

Q25. Where are sample images stored?

A. In the `assets/` directory: `assets/sample.jpg`. This is a pre-packaged terrain image that demonstrates the app's capabilities without requiring users to find their own images.

7. Configuration & State

Q26. What is `APP_CONFIG` and how is it used?

A.

```
APP_CONFIG = {
    "system_os": platform.system(),          # "Darwin", "Linux", etc.
    "python_version": sys.version.split()[0], # "3.10.12"
    "torch_version": torch.__version__,      # "2.1.0"
```

```
"device": "cuda" if torch.cuda.is_available() else "cpu"
}
```

This dictionary captures system info at startup and is passed to `wandb.init(config=APP_CONFIG)` so every W&B run records the environment it was run on. Useful for debugging performance differences across machines.

Q27. How is the `MODELS` dictionary structured?

A.

```
MODELS = {
    "SegFormer B0 (Fast)": "nvidia/segformer-b0-finetuned-ade-512-512",
    "SegFormer B2 (Balanced)": "nvidia/segformer-b2-finetuned-ade-512-512"
}
```

Keys are **human-readable names** shown in the dropdown. Values are **HuggingFace model IDs** used for downloading/loading. This decouples the UI labels from the actual model identifiers.

Q28. How is the default mapping defined?

A.

```
DEFAULT_MAPPING = {
    "safe": model_utils.SAFE_LABELS_DEFAULT,
    "hazard": model_utils.HAZARD_LABELS_DEFAULT
}
```

It references the keyword lists from `model_utils.py` and is serialized to JSON for the code editor's default value:

```
mapping_editor = gr.Code(
    value=json.dumps(DEFAULT_MAPPING, indent=2), # Pretty-printed JSON string
    ...
)
```

8. W&B Integration in the Frontend

Q29. How is W&B initialized in the app?

A.

```
INFERENCE_TABLE = None
try:
    wandb.init(project="terrain-safety-v1", job_type="inference", config=APP_CONFIG)
    INFERENCE_TABLE = wandb.Table(columns=[
        "model", "score", "safe_pct", "hazard_pct",
        "time_ms", "top_class", "confidence", "image_ref"
    ])
except Exception as e:
    print(f"Warning: W&B init failed: {e}. Logging disabled.")
```

- Runs at **import time** (module level), so a single W&B run spans the app's lifetime.
- If `WANDB_API_KEY` is missing or W&B is unreachable, the `except` block catches the error and logging is silently disabled. The app still functions normally.

- `INFERENCE_TABLE` is a global `wandb.Table` that accumulates one row per inference call.

Q30. How does the callback log to W&B?

A.

```
try:
    meta = {
        "model_id": model_name,
        "inference_time_ms": inference_time_ms,
        "image_size": image.size,
        "device": device,
        "depth_enabled": enable_depth,
        "path_enabled": enable_path
    }
    model_utils.log_inference_to_wandb(image, hud_image, stats, meta, table=INFERENCE_TABLE)
except Exception as e:
    print(f"W&B Log failed: {e}")
```

Wrapped in try-except so a logging failure never crashes the user-facing app.

9. Deployment Specifics

Q31. How do you deploy on Hugging Face Spaces?

A.

1. Create a new Space (SDK: Gradio, Hardware: CPU Basic).
2. Upload `app.py` , `model_utils.py` , `requirements.txt` .
3. Optionally add `WANDB_API_KEY` as a Space secret.
4. The Space auto-builds, installs deps, and runs `python app.py` .

Q32. What does `.queue()` do?

A. `demo.queue()` enables **request queueing**. Without it, if two users click "Analyze" simultaneously, one request might fail or interfere. With queueing, requests are serialized — each waits in a FIFO queue and is processed one at a time. This is critical for GPU-bound inference where concurrent model calls could cause OOM errors.

Q33. What does `launch(share=False)` vs `launch(share=True)` do?

A.

- `share=False` → Local server only at `http://127.0.0.1:7860` . Only accessible from the same machine.
- `share=True` → Gradio creates a **temporary public URL** (e.g., `https://xxxx.gradio.live`) via a tunnel. Useful for demos — anyone with the link can access the app. The URL expires after 72 hours.

Q34. What port does the app use?

A. Default Gradio port is **7860**. Can be changed with `demo.launch(server_port=8080)` .

10. Error Handling in the UI

Q35. How does the app handle a failed model load?

A.

```
feature_extractor, model, device = model_utils.load_model(model_name)
if model is None:
```

```
return None, None, None, "Error loading model."
```

Returns an error string to the JSON output, and `None` for all visual outputs. The Gradio UI shows blank image boxes and the error message.

Q36. How is "No Safe Path Found" displayed?

A.

```
if enable_path and path_coords is None:
    draw = PIL.ImageDraw.Draw(hud_image)
    draw.text((20, 20), "No Safe Path Found", fill=(255, 0, 0))
```

Red text is drawn directly onto the HUD image at coordinates (20, 20). This happens when pathfinding was enabled but the algorithm couldn't find a valid path (cost too high or no safe pixels).

Q37. What happens if the user enters invalid JSON in the mapping editor?

A. The bare `except` clause catches `json.JSONDecodeError` and silently falls back to `DEFAULT_MAPPING`. The app continues processing with default safety definitions — no crash, no error shown. A more robust version could return a warning message.

11. Gradio Component Deep Dive

Q38. What is `gr.Blocks` vs `gr.Interface` ?

A.

- **`gr.Interface`** : Simple wrapper — one function, one set of inputs, one set of outputs. Good for single-model demos.

```
gr.Interface(fn=classify, inputs="image", outputs="label").launch()
```

- **`gr.Blocks`** : Full layout control — multiple components, custom arrangement, multiple event handlers, accordions, tabs, state management. Our app needs this for its complex multi-section layout with accordions and optional features.

Q39. Can `gr.Blocks` have multiple buttons with different callbacks?

A. Yes! Each button can have its own `.click()` with different `fn`, `inputs`, and `outputs`. Our app uses a single button, but you could add separate "Quick Analyze" and "Deep Analyze" buttons with different functions.

Q40. What is `gr.Row()` vs `gr.Column()` ?

A.

- `gr.Row()` → Places children **side by side** (horizontal layout).
- `gr.Column()` → Places children **stacked** (vertical layout).

In our app:

```
with gr.Row():
    # Top-level horizontal split
    with gr.Column(scale=1):
        # Left column (inputs)
    with gr.Column(scale=2):
        # Right column (outputs)
        with gr.Row():
            # Sub-row for 3 images side by side
            output_hud = gr.Image(...)
            output_depth = gr.Image(...)
            output_mask = gr.Image(...)
```

Q41. What is `variant="primary"` on the button?

A.

```
run_btn = gr.Button("Analyze Terrain", variant="primary")
```

`variant="primary"` gives the button **highlighted styling** (usually colored/bold) to distinguish it as the main action. Other options: `"secondary"` (muted), `"stop"` (red/destructive).

12. Timing & Performance in the Frontend

Q42. How is inference time measured?

A.

```
import time
start_time = time.time()
mask, logits = model_utils.predict_mask(image, (...))
end_time = time.time()
inference_time_ms = round((end_time - start_time) * 1000, 2)
```

Uses Python's `time.time()` for wall-clock timing. Only measures the **segmentation inference** step — not depth, pathfinding, or visualization. The time is displayed in W&B logs and the stats JSON.

Q43. What latency can a user expect?

A.

- **B0 on CPU:** ~100–300ms per image.
- **B0 on GPU:** <50ms.
- **Depth estimation** adds ~200–500ms (CPU) or ~50ms (GPU).
- **Pathfinding** adds ~50–200ms depending on image size.
- **3D rendering** adds ~100ms for figure generation.

Total end-to-end with all features on CPU: ~500ms–1.5s.

13. Credentials & Environment

Q44. How are API keys managed?

A.

```
from dotenv import load_dotenv
load_dotenv()
```

`python-dotenv` reads the `.env` file at startup and sets environment variables. The `.env` file (gitignored) contains:

```
WANDB_API_KEY=your_key_here
```

W&B automatically reads `WANDB_API_KEY` from the environment. For Hugging Face Spaces, keys are set as **Space Secrets** in the Settings UI.

Q45. What is `.env.example` ?

A. A template file (committed to git) showing what environment variables are needed, without real values:

```
WANDB_API_KEY=your_key_here
HF_TOKEN=optional_if_needed
```

Users copy it to `.env` and fill in their real keys. The `.env` file is in `.gitignore` to prevent accidental credential leaks.

14. Rapid Fire

Q46. What Python version is required?

A. Python 3.10+ (as per README badge).

Q47. How many input components does the UI have?

A. 10 — 1 image, 2 dropdowns, 2 sliders, 3 checkboxes, 1 code editor, 1 button.

Q48. How many output components?

A. 6 — 3 images, 1 plot, 1 label, 1 JSON viewer.

Q49. What does `demo.queue()` return?

A. Returns the same `demo` object (chainable), so you can write `demo.queue().launch()`.

Q50. Can Gradio handle file uploads other than images?

A. Yes — `gr.File`, `gr.Audio`, `gr.Video` etc. Our app only uses `gr.Image`.

Q51. What is the `title` parameter in `gr.Blocks`?

A. Sets the **browser tab title**: `gr.Blocks(title="Terrain Safety Analysis")` → the tab reads "Terrain Safety Analysis".

Q52. How is the heading added to the UI?

A.

```
gr.Markdown("# 🚧 Terrain Safety Analysis with SegFormer (v2)")
gr.Markdown("Upload a terrain image to analyze Safe vs Hazard regions boundaries.")
```

`gr.Markdown` renders markdown text in the UI. The `#` creates an H1 heading. Supports emojis, bold, links, etc.

Q53. Can users interact with the 3D plot?

A. Yes — `gr.Plot` with a Plotly figure supports **rotation, zoom, pan, and hover tooltips** natively in the browser. No extra code needed.

Q54. What happens if W&B is not configured?

A. The `wandb.init()` call fails gracefully, `INFERENCE_TABLE` stays `None`, and `log_inference_to_wandb()` returns immediately when `wandb.run` is `None`. The app works perfectly without W&B — logging is entirely optional.

Q55. How would you add a new model variant to the dropdown?

A. Add one line to the `MODELS` dictionary:

```
MODELS = {
    "SegFormer B0 (Fast)": "nvidia/segformer-b0-finetuned-ade-512-512",
    "SegFormer B2 (Balanced)": "nvidia/segformer-b2-finetuned-ade-512-512",
```

```
"SegFormer B5 (Accurate)": "nvidia/segformer-b5-finetuned-ade-640-640", # NEW  
}
```

The dropdown automatically picks up the new entry. No other code changes needed thanks to the data-driven design.